Mini project report on

Operating System

Submitted in partial fulfilment of the requirements for the award of degree of

Bachelor of Technology

in

Computer Science & Engineering

UE24CS242B - Operating Systems

Submitted by:

Pratham Kumar	**PES2UG24CS368**
Karthik P	**PES2UG24CS348**

under the guidance of

Prof. Neha Sharma

Assistant Professor

PES University

Jan- May 2026

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

FACULTY OF ENGINEERING

PES UNIVERSITY

(Established under Karnataka Act No. 16 of 2013)

Electronic City, Hosur Road, Bengaluru – 560 100, Karnataka, India

PES UNIVERSITY

(Established under Karnataka Act No. 16 of 2013)

Electronic City, Hosur Road, Bengaluru – 560 100, Karnataka, India

CERTIFICATE

This is to certify that the mini project entitled

Operating Systems

is a bonafide work carried out by

Pratham Kumar	**PES2UG24CS368**
Karthik P	**PES2UG24CS348**

In partial fulfilment for the completion of forth semester OS Project (UE24CS242B) in the Program of Study - Bachelor of Technology in Computer Science and Engineering under rules and regulations of PES University, Bengaluru during the period Jan 2026 – May 2026. It is certified that all corrections / suggestions indicated for internal assessment have been incorporated in the report. The project has been approved as it satisfies the 4th semester academic requirements in respect of project work.

Signature

Prof. Neha Sharma

Assistant Professor

DECLARATION

We hereby declare that the DBMS Project entitled **Operating System** has been carried out by us under the guidance of **Prof. Neha Sharma, Assistant Professor** and submitted in partial fulfilment of the course requirements for the award of degree of **Bachelor of Technology** in **Computer Science and Engineering** of **PES University, Bengaluru** during the academic semester Jan – May 2026.

Pratham Kumar	**PES2UG24CS368**	**<Signature>**
Karthik P	**PES2UG24CS348**	**<Signature>**

ACKNOWLEDGEMENT

I would like to express my gratitude to Prof. Neha Sharma, Department of Computer Science and Engineering, PES University, for her continuous guidance, assistance, and encouragement throughout the development of this UE24CS242B - Operating Systems Project.

I take this opportunity to thank Dr. Sandesh B J, C, Professor, Chairperson, Department of Computer Science and Engineering, PES University, for all the knowledge and support I have received from the department.

I am deeply grateful to Prof. Jawahar Doreswamy, Chancellor – PES University, Dr. Suryaprasad J, Vice-Chancellor, PES University for providing to me various opportunities and enlightenment every step of the way. Finally, this DBMS Project could not have been completed without the continual support and encouragement I have received from my family and friends.

ABSTRACT

This project presents the design and implementation of a lightweight container runtime system named “Jackfruit”, developed using the C programming language on a Linux-based environment. The primary objective of the project is to gain a deep understanding of core Operating System concepts by building a simplified version of containerization technology from scratch.

The system enables the creation and management of isolated execution environments, referred to as containers, by leveraging low-level Linux system calls such as clone(), chroot(), sethostname(), and exec(). These containers operate with separate process IDs, hostnames, and filesystem views, thereby demonstrating process and resource isolation using Linux namespaces (PID, UTS, and mount namespaces).

A key component of the project is the implementation of a custom Linux kernel module for memory monitoring and enforcement. The module interacts with user-space programs through a character device interface (/dev/container_monitor) and uses ioctl-based communication to track memory usage of containerized processes. It enforces both soft and hard memory limits, issuing warnings or terminating processes when limits are exceeded, thereby illustrating kernel-level control over resource management.

The runtime provides a command-line interface (CLI) that supports essential container operations such as start, stop, run, logs, and process listing (ps). It also implements a logging mechanism using inter-process communication (pipes) to capture and store container output efficiently.

To demonstrate system behavior, three distinct workloads were developed: a CPU-intensive program (cpu_hog), an I/O-intensive program (io_pulse), and a memory-intensive program (memory_hog). These workloads were executed inside containers to analyze scheduling behavior, I/O handling, and memory allocation patterns under constrained environments.

Throughout the project, several practical challenges such as environment setup, network configuration in virtual machines, file system management, and kernel module integration were encountered and resolved. The successful implementation of this system provides a comprehensive understanding of containerization, process lifecycle management, memory monitoring, and the interaction between user space and kernel space.

Overall, the project effectively bridges theoretical Operating System concepts with practical implementation, offering valuable insights into how modern container technologies function at a low level.

TABLE OF CONTENTS

CHAPTER NO.	TITLE	PAGE NO.
1.	INTRODUCTION	9 - 12
2.	PROBLEM DEFINITION	13 - 15
3.	METHODOLOGY	16 - 19
4.	CODE	20 - 34
5.	RESULT AND DISCUSSION	35 - 37
6.	OUTPUTS	38 – 41
7.	GITHUB LINK	42 – 44
REFERENCES/BIBLIOGRAPHY		45
APPENDIX A DEFINITIONS, ACRONYMS AND ABBREVIATIONS		46 - 48

LIST OF FIGURES (Output Screenshots)

FIGUER NO.	TITLE OF SCREENSHOT	IMAGE FILE NAMED IN GITHUB	PAGE NO.
FIG 6.1	SUCCESSFUL BUILD OF PROJECT USING MAKEFILE	SCREENSHOTS_1_BUILD_SUCCESS.PNG	38
FIG 6.2	ROOT FILESYSTEM SETUP USING ALPINE LINUX	SCREENSHOTS_2_ROOTFS_SETUP.PNG	39
FIG 6.3	KERNEL MODULE LOADING AND DEVICE CREATION	SCREENSHOTS_3_KERNEL_MODULE.PNG	39
FIG 6.4	SUPERVISOR INITIALIZATION AND RUNTIME SETUP	SCREENSHOTS_4_SUPERVISOR_READY.PNG	39
FIG 6.5	EXECUTION OF CPU-INTENSIVE WORKLOAD (CPU_HOG)	SCREENSHOTS_5_CPU_HOG.PNG	40
FIG 6.6	EXECUTION OF I/O-INTENSIVE WORKLOAD (IO_PULSE)	SCREENSHOTS_6_IO_PULSE.PNG	40
FIG 6.7	EXECUTION OF MEMORY-INTENSIVE WORKLOAD (MEMORY_HOG)	SCREENSHOTS_7_MEMORY_HOG.PNG	40
FIG 6.8	CONTAINER CONTROL COMMANDS (START, STOP, PS, LOGS)	SCREENSHOTS_8_CONTROL_RUN.PNG	41

INTRODUCTION

An Operating System (OS) is a fundamental system software that acts as an interface between the user and the computer hardware. It plays a crucial role in managing system resources and ensuring that application programs execute efficiently and securely. The OS is responsible for handling core functionalities such as process management, memory management, file system organization, device management, and system security. Without an operating system, users would have to directly interact with hardware, which would be complex and inefficient.

One of the primary responsibilities of an operating system is process management. It controls the creation, execution, and termination of processes, ensuring that multiple programs can run concurrently without interfering with each other. The OS uses scheduling algorithms to allocate CPU time among processes, optimizing performance and responsiveness. Memory management is another critical function, where the OS allocates and deallocates memory dynamically, ensuring efficient utilization of RAM while preventing conflicts between processes.

The operating system also manages file systems, enabling users to store, retrieve, and organize data efficiently. It provides abstraction for storage devices and ensures data integrity and access control. Device management is handled through drivers, allowing the OS to communicate with hardware components such as keyboards, disks, and network interfaces. Additionally, the OS provides mechanisms for inter-process communication (IPC), enabling processes to exchange data and coordinate execution.

In modern computing environments, operating systems have evolved to support advanced features such as virtualization and containerization. Containers, in particular, have become widely used due to their lightweight nature and efficient resource utilization. Unlike traditional virtual machines, containers share the host OS kernel while maintaining isolation, making them faster and more scalable. This concept is fundamental to cloud computing and modern software deployment practices.

The Operating Systems course (UE24CS242B) focuses on understanding these core principles and how they are implemented in real-world systems. The course covers topics such as process scheduling, synchronization, deadlocks, memory management techniques like paging and segmentation, file systems, and kernel architecture. It also introduces practical aspects of system programming, where students interact directly with system calls and low-level OS mechanisms. Through this course, students gain both theoretical knowledge and hands-on experience in building and analyzing system-level components.

In this project, several important operating system concepts have been applied and demonstrated in a practical manner. One of the key concepts is process isolation using namespaces. Namespaces are a feature of the Linux kernel that provide isolation of system resources such as process IDs (PID namespace), hostnames (UTS namespace), and mount points (mount namespace). By using these namespaces, processes running inside a container are isolated from the host system and from other containers.

Another important concept used in this project is filesystem isolation using the chroot() system call. The chroot() call changes the apparent root directory for a process, ensuring that it cannot access files outside its designated environment. This is essential for creating secure and isolated container environments. Along with this, the project makes use of system calls such as clone(), which is used to create new processes with specific namespace configurations, and exec(), which replaces the current process image with a new program.

Inter-process communication (IPC) is also a major aspect of this project. Pipes are used to capture and redirect the output (stdout and stderr) of container processes to log files. This allows the system to maintain a record of container execution and provides a mechanism for monitoring behavior. Synchronization mechanisms such as mutexes and condition variables are used to ensure safe access to shared resources, preventing race conditions in concurrent environments.

Memory management is another critical concept demonstrated in this project. A custom Linux kernel module is developed to monitor the memory usage of containerized processes. The module tracks the Resident Set Size (RSS) of processes and enforces predefined soft and hard memory limits. When a process exceeds the soft limit, a warning is generated, and when it exceeds the hard limit, the process is terminated using a SIGKILL signal. This demonstrates how the operating system enforces resource constraints and maintains system stability.

Scheduling behavior is analyzed using different workloads. CPU-bound workloads continuously consume processor time, while I/O-bound workloads frequently perform input/output operations. By running these workloads inside containers, the project demonstrates how the OS scheduler allocates CPU time and manages different types of processes. This provides insight into scheduling policies and system performance.

The project titled "OS Jackfruit Container Runtime and Memory Monitor" is a practical implementation of these concepts. It involves designing a lightweight container runtime system using the C programming language. The runtime allows users to create, manage, and monitor containers through a command-line interface. Each container runs in an isolated environment with its own root filesystem and namespace configuration.

The runtime includes a supervisor process that acts as the central controller for all containers. It manages container lifecycle events such as creation, execution, termination, and logging. The system supports commands such as start, stop, run, logs, and ps, allowing users to interact with containers easily. The logging mechanism captures the output of containers and stores it for later analysis.

In addition to the runtime, a kernel module is integrated into the system to provide memory monitoring and enforcement capabilities. This module interacts with user-space programs through a device file and uses ioctl-based communication to exchange information. It continuously monitors memory usage and ensures that containers do not exceed predefined limits.

To validate the system, three types of workloads were implemented: a CPU-intensive program (cpu_hog), an I/O-intensive program (io_pulse), and a memory-intensive program (memory_hog). These workloads were executed inside containers to observe system behavior under different conditions. The results demonstrate that the system successfully isolates processes, captures logs, and enforces memory constraints.

Overall, this project provides a comprehensive understanding of operating system concepts by combining theory with practical implementation. It highlights the interaction between user space and kernel space, the importance of process isolation, and the mechanisms used by modern systems to manage resources efficiently. The project serves as a foundation for understanding advanced container technologies and system-level programming.

PROBLEM DEFINITION

The primary objective of this project is to design and implement a lightweight container runtime system along with a kernel-level memory monitoring module, in order to understand and demonstrate fundamental operating system concepts through practical implementation. The project, referred to as the “Jackfruit OS Mini Project”, is structured into multiple tasks that collectively simulate the working of modern container systems such as Docker, but at a much lower level using system programming in C.

The problem statement focuses on building a system that can create isolated execution environments (containers), manage their lifecycle, capture their outputs, and enforce memory constraints through kernel-level monitoring. Unlike high-level container tools, this project requires implementing the core functionality manually using Linux system calls and kernel programming techniques.

The project is divided into six major tasks, each addressing a specific aspect of operating system design and implementation.

Task 1 involves setting up the environment and building the base system. This includes installing the required dependencies, compiling the user-space runtime and kernel module using a Makefile, and ensuring that all components build successfully without errors. The challenge in this task lies in correctly configuring the development environment, resolving compilation issues, and understanding the build process for both user-space and kernel-space programs.

Task 2 focuses on creating isolated root filesystems for containers. In this step, a minimal Linux filesystem (Alpine Linux) is downloaded and extracted. Multiple copies of this base filesystem are created to serve as independent environments for different containers. The problem here is to ensure proper filesystem isolation, so that each container operates

independently without interfering with others or the host system. This task also introduces the concept of chroot(), which is used to change the root directory of a process.

Task 3 involves implementing and loading the kernel module for memory monitoring. The module is responsible for tracking memory usage of container processes and enforcing limits. It creates a device file (/dev/container_monitor) through which user-space programs communicate using ioctl calls. The challenge in this task is understanding kernel programming concepts, handling device registration, and ensuring safe interaction between kernel space and user space.

Task 4 focuses on implementing the container runtime and supervisor mechanism. The runtime uses system calls such as clone() to create new processes with namespace isolation. The supervisor acts as a central controller that manages all containers, maintains metadata, and processes user commands. The problem here is to correctly implement process creation, maintain parent-child relationships, and ensure proper cleanup of processes to avoid zombie states.

Task 5 deals with workload execution and logging. Different types of workloads (CPU-intensive, I/O-intensive, and memory-intensive) are executed inside containers. The runtime captures the output of these workloads using pipes and redirects them to log files. This task demonstrates inter-process communication and synchronization. The main challenge is to ensure that logging is handled efficiently without losing data or causing race conditions.

Task 6 involves memory monitoring and enforcement. The kernel module continuously tracks the memory usage of container processes and enforces soft and hard limits. When the soft limit is exceeded, a warning is generated, and when the hard limit is exceeded, the process is terminated using a signal. This task highlights the role of the operating system in resource management and protection. The challenge lies in accurately tracking memory usage and safely terminating processes when required.

In addition to these tasks, the project also requires implementing a command-line interface (CLI) that allows users to interact with the system. Commands such as start, stop, run, ps, and logs must be implemented to manage containers effectively. This adds another layer of complexity, as the system must handle multiple concurrent operations and maintain consistency in container states.

The submission requirements for this project include multiple components. Firstly, the complete source code must be uploaded to a GitHub repository, ensuring that only necessary files are included and unnecessary files such as compiled binaries or temporary files are excluded. The repository must contain properly structured directories, clear documentation, and a detailed README file explaining the project, setup, execution steps, and results.

Secondly, the submission must include a comprehensive report following a specific format. The report should contain sections such as Introduction, Problem Definition, Methodology, Code Explanation, Results and Discussion, Outputs, and References. It must provide a detailed explanation of the implementation, challenges faced, and solutions adopted.

Thirdly, the submission requires capturing screenshots as proof of execution for each task. These screenshots must demonstrate successful build, root filesystem setup, kernel module loading, supervisor execution, workload execution, memory enforcement, and command-line operations. Each screenshot must be properly labeled and included in the report.

Overall, the problem defined in this project is not limited to simply writing code, but involves understanding and integrating multiple operating system concepts into a working system. It requires a combination of theoretical knowledge and practical skills, including system programming, kernel module development, debugging, and documentation. The successful completion of this project demonstrates a deep understanding of how operating systems manage processes, memory, and system resources in real-world environments.

METHODOLOGY

The methodology followed in this project involves a systematic approach starting from environment setup to the final execution and verification of all tasks. The implementation was carried out in a Linux-based environment using system-level programming in C, along with kernel module development. The entire process can be divided into multiple stages, each addressing a specific requirement of the project.

The first step in the methodology was setting up the development environment. A virtual machine was created using VMware Workstation, and Ubuntu Linux was installed as the operating system. This environment was chosen because Linux provides direct access to system calls and kernel-level functionalities required for implementing containerization and kernel modules. During the setup, issues related to network connectivity were encountered, which were resolved by configuring DHCP settings and restarting network services. Required packages such as build-essential, Linux kernel headers, and wget were installed to support compilation and system programming.

After setting up the environment, the next step involved obtaining the project source code from the provided GitHub repository. The repository was cloned into the Ubuntu system, and the directory structure was examined to understand the organization of files, including the boilerplate code, Makefile, and workload programs. Care was taken to ensure that only relevant files were present in the working directory, and unnecessary files were removed or ignored using the .gitignore configuration.

The next phase focused on building the project components. The Makefile provided in the boilerplate directory was used to compile both user-space programs and the kernel module. The command “make clean” was executed to remove any previous build artifacts, followed by “make” to compile the project. This step generated the runtime executable (engine), the kernel module (monitor.ko), and workload binaries such as cpu_hog, memory_hog, and io_pulse.

During compilation, compatibility issues related to kernel functions were identified and resolved to ensure successful build.

Following the build process, the root filesystem setup was performed. A minimal Alpine Linux root filesystem was downloaded using wget and extracted into a base directory. This base filesystem served as the foundation for all containers. Multiple copies of this root filesystem were created, such as rootfs-alpha, rootfs-beta, and rootfs-gamma, to provide isolated environments for different containers. The workload binaries were then copied into the respective root filesystems to ensure that each container could execute the required program. This step was crucial for enabling filesystem isolation using the chroot() system call.

The next stage involved loading the kernel module responsible for memory monitoring. The compiled module (monitor.ko) was inserted into the kernel using the insmod command with superuser privileges. The successful loading of the module was verified using lsmod, and the creation of the device file (/dev/container_monitor) was confirmed. This device file serves as the communication interface between user-space programs and the kernel module. Kernel logs were also checked using dmesg to ensure that the module was functioning correctly.

After setting up the kernel module, the container runtime was executed. The runtime operates in two modes: supervisor mode and client mode. In supervisor mode, the runtime initializes a long-running process that manages all containers and listens for incoming commands through a control socket. The supervisor maintains metadata for each container, including its ID, process ID, state, and execution details.

In client mode, commands such as start, run, stop, ps, and logs are issued to interact with the supervisor. The start command is used to create a new container, specifying its ID, root filesystem, and the program to execute. The runtime uses the clone() system call with namespace flags to create a new process that operates in an isolated environment. The namespaces ensure separation of process IDs, hostnames, and mount points. After process creation, the chroot() system call is used to change the root directory of the container,

restricting its access to the specified filesystem. The exec() call is then used to execute the workload program within the container.

To capture the output of container processes, a logging mechanism was implemented using pipes. The standard output and standard error of the container process were redirected to a pipe using dup2(). A separate logging component reads from the pipe and writes the output to log files. This ensures that all container activity is recorded and can be accessed later using the logs command. Synchronization mechanisms such as mutexes were used to prevent race conditions while accessing shared resources.

The next stage involved executing different workloads inside containers to analyze system behavior. The cpu_hog program was used to generate continuous CPU load, demonstrating how the scheduler allocates processor time. The io_pulse program generated periodic output, allowing observation of I/O operations and logging behavior. The memory_hog program incrementally allocated memory to test the monitoring capabilities of the kernel module. Each workload was executed using the start command, and its behavior was observed using the ps and logs commands.

The most critical part of the methodology was testing memory monitoring and enforcement. The kernel module continuously tracked the memory usage of container processes by monitoring their resident set size (RSS). When the memory usage exceeded a predefined soft limit, a warning was generated. When the hard limit was exceeded, the module terminated the process using a SIGKILL signal. This behavior was verified by observing the termination of the memory_hog container in the process list, confirming that memory enforcement was functioning correctly.

In addition to workload execution, lifecycle management of containers was also tested. The stop command was used to terminate running containers, and the run command was used to execute containers in foreground mode. The ps command provided a summary of all containers

and their states, while the logs command displayed their output. These operations demonstrated the correct functioning of the command-line interface and the supervisor mechanism.

The final stage of the methodology involved documentation and validation. Screenshots were captured at each step, including build success, root filesystem setup, kernel module loading, supervisor execution, workload execution, memory enforcement, and command-line operations. These screenshots served as proof of successful implementation and were included in the report.

Finally, the project repository was prepared for submission. All relevant source code files were added to the Git repository, while unnecessary files such as compiled binaries, temporary files, and logs were excluded using the .gitignore file. The changes were committed and pushed to the remote repository on GitHub. A detailed README file was created to document the project, including setup instructions, execution steps, and explanations of the implemented features.

Overall, the methodology followed in this project ensured a structured and systematic implementation of all required components. Each step was carefully executed, tested, and validated, resulting in a fully functional container runtime system with integrated memory monitoring capabilities. This approach not only helped in completing the project successfully but also provided a deep understanding of operating system concepts and their practical applications.

CODE

The implementation of this project is mainly divided into two core source files: engine.c and monitor.c. The file engine.c is a user-space program that implements the lightweight container runtime and supervisor, whereas monitor.c is a kernel-space module that monitors memory usage of the container processes and enforces memory constraints. Together, these two files form the complete runtime system.

The file engine.c acts as the central controller of the project. It is responsible for creating containers, managing their execution, tracking metadata, handling logs, and communicating with the kernel module. In other words, engine.c is the user-space runtime through which all container-related operations are performed.

The file monitor.c acts as the memory monitoring component in kernel space. It registers itself as a Linux kernel module, creates a device file, accepts monitoring requests from engine.c, periodically checks memory usage of registered processes, and takes action when a process exceeds configured memory limits.

The overall design separates control logic and enforcement logic. The runtime file manages container creation and user commands, while the kernel module manages low-level memory observation and enforcement. This separation reflects an important operating system principle: user space handles policy and interaction, while kernel space handles protected resource control.

4.1 ENGINE.C — USER-SPACE RUNTIME

The engine.c file is the largest and most important user-space component of the project. It implements the supervisor, container creation, command handling, metadata maintenance, and logging support.

The runtime supports multiple commands such as supervisor, start, run, ps, logs, and stop. These commands allow the user to interact with the container system through a command-line interface.

A simplified glimpse of the command-type structure is shown below:

```
typedef enum {
   CMD_SUPERVISOR = 0,
   CMD_START,
   CMD_RUN,
   CMD_PS,
   CMD_LOGS,
   CMD_STOP
} command_kind_t;
```

This enumeration is important because it defines the different operations that the runtime can perform. When the user executes the runtime with a specific command, the command is identified through this structure and the corresponding handler is executed.

The runtime also keeps track of the lifecycle of each container. For that purpose, container states are defined as follows:

```
typedef enum {
   CONTAINER_STARTING = 0,
   CONTAINER_RUNNING,
   CONTAINER_STOPPED,
   CONTAINER_KILLED,
   CONTAINER_EXITED
} container_state_t;
```

These states help the supervisor understand whether a container is still starting, currently running, stopped by user request, killed by policy, or exited normally. This state information is later shown in the ps command output.

To maintain complete information about every container, the runtime uses a metadata structure. A simplified glimpse is shown below:

```
typedef struct container_record {
    char id[CONTAINER_ID_LEN];
    pid_t host_pid;
    time_t started_at;
    container_state_t state;
    unsigned long soft_limit_bytes;
    unsigned long hard_limit_bytes;
    int exit_code;
    int exit_signal;
    char log_path[PATH_MAX];
    struct container_record *next;
} container_record_t;
```

This structure is extremely important because it stores the identity and status of every container managed by the supervisor. It contains:

- container ID
- host process ID
- start time
- state
- memory limits
- exit information
- path to the corresponding log file

The supervisor maintains these records as a linked list, which allows multiple containers to be tracked dynamically.

Another major part of engine.c is the supervisor context structure. It stores global runtime resources such as the control socket, monitor device descriptor, logging thread, metadata lock, and container list. This context makes it possible to manage the entire system from one long-running process.

When the runtime is started in supervisor mode, it initializes its control structures and creates a command interface. The supervisor remains active and waits for incoming requests. All other commands, such as start or stop, behave like short-lived clients that connect to the supervisor, send a request, receive a response, and terminate.

This design is important because it creates a clear separation between the long-running control process and user-issued commands. It also reflects a practical OS design where one central controller manages many child processes.

One of the most critical operations in engine.c is container creation. This is done using the clone() system call. The clone call is used instead of a normal fork because clone allows namespace isolation.

A simplified glimpse is:

```
clone(child_fn, stack_top,
    CLONE_NEWPID | CLONE_NEWUTS | CLONE_NEWNS | SIGCHLD,
    cfg);
```

This line is conceptually important. It means that the child process is created with:
- a new PID namespace

- a new UTS namespace
- a new mount namespace
- normal SIGCHLD delivery to the parent

Each of these flags provides a different level of isolation.

The child process then enters the child function, where the actual container environment is prepared. The child function performs several critical steps:

1. redirects output
2. sets hostname
3. changes root filesystem
4. mounts /proc
5. executes the workload

A conceptual glimpse of this logic is:

```
if (dup2(cfg->log_write_fd, STDOUT_FILENO) < 0) { ... }
if (dup2(cfg->log_write_fd, STDERR_FILENO) < 0) { ... }

sethostname(cfg->id, strlen(cfg->id));

if (chroot(cfg->rootfs) != 0) { ... }
if (chdir("/") != 0) { ... }

mount("proc", "/proc", "proc", 0, NULL);

execl(cfg->command, cfg->command, NULL);
```

The first part redirects stdout and stderr to the logging system, which ensures that all output generated by the workload is captured by the runtime. The sethostname call gives the container its own hostname, which demonstrates UTS namespace isolation.

The chroot() call changes the root directory of the process so that the process sees the container root filesystem as its own /. This is essential for filesystem isolation. The chdir("/") call ensures that the process starts execution from the new root.

The mount of /proc is necessary because many process-related tools inside a container depend on the proc filesystem. Without mounting /proc, commands that inspect process information may fail or behave incorrectly.

Finally, execl() replaces the child process image with the selected workload such as /cpu_hog, /io_pulse, or /memory_hog.

After clone() returns in the parent, the supervisor stores container metadata and updates the container state. The child process is now running as a managed container.

Another major part of engine.c is the command-line interface implementation. The runtime accepts commands like:
- start
- run
- stop
- ps
- logs

The start command launches a container in the background. The run command launches a container and waits for it to finish. The stop command sends a stop request to a running container. The ps command displays metadata of all tracked containers. The logs command shows the collected output from the corresponding log file.

The difference between start and run is important. In start mode, the user receives control immediately after the request is accepted. In run mode, the runtime blocks until the container exits. This allows the runtime to support both background and foreground execution models.

Logging is another important subsystem in engine.c. Container output is captured through pipes and stored through a bounded-buffer design. A bounded buffer is used so that log producers and log consumers can work safely without corrupting shared state.

The buffer structure conceptually looks like:

```
typedef struct {
    log_item_t items[LOG_BUFFER_CAPACITY];
    size_t head;
    size_t tail;
    size_t count;
    int shutting_down;
    pthread_mutex_t mutex;
    pthread_cond_t not_empty;
    pthread_cond_t not_full;
} bounded_buffer_t;
```

This structure supports classic producer-consumer synchronization. Producers add new log items into the queue, while a consumer removes them and writes them to a file. The mutex protects shared state, and the condition variables coordinate the “buffer full” and “buffer empty” cases.

This design is important because direct uncontrolled writes from multiple contexts could cause race conditions or corrupted logs. The bounded-buffer model ensures that logging remains structured and synchronized.

The runtime also includes child reaping logic. When a child exits, the supervisor must call waitpid() to collect its termination status and avoid zombie processes. This is a key operating system concept. The runtime updates the metadata depending on whether the container exited normally or was killed by a signal.

For example:
- if the process ends normally, exit_code is stored
- if the process receives SIGKILL, exit_signal is stored and state becomes KILLED

This behavior is clearly visible in the ps output of the project.

Finally, engine.c also communicates with the kernel module through the monitor device. Once a container is created, the runtime registers its PID and memory limits with the kernel module. This allows the kernel to begin monitoring the process.

In summary, engine.c implements:
- command parsing
- supervisor logic
- container creation
- namespace isolation
- root filesystem switching
- logging
- metadata tracking
- child reaping
- kernel module registration

Because of this, engine.c can be considered the complete user-space runtime manager of the project.

4.2 MONITOR.C — KERNEL-SPACE MEMORY MONITOR

The monitor.c file implements the Linux kernel module used to monitor memory usage of container processes. It runs in kernel space and is loaded using insmod. Once loaded, it creates a device file through which user-space programs can communicate with it.

The purpose of this module is to provide memory observation and enforcement. A user-space runtime alone can observe processes, but enforcement is more reliable in kernel space because the kernel already has direct authority over process state and resource accounting.

The module first includes the required Linux kernel headers. These headers provide access to character devices, memory management APIs, timers, work queues, and user-kernel data transfer.

The module uses a structure to store information about each monitored process. This structure typically contains:
- process ID
- soft limit
- hard limit
- whether soft warning has been issued
- linked-list pointer

Conceptually, the module maintains a linked list of monitored entries. Each time engine.c creates a container, it registers the corresponding PID with this module.

A shared header file, monitor_ioctl.h, defines the ioctl interface between engine.c and monitor.c. This file is crucial because both user space and kernel space must agree on the exact data format and command identifiers.

The kernel module creates a character device, which becomes visible as /dev/container_monitor. This is done during module initialization. The init function performs the following tasks:

1. allocates character device number
2. initializes device class
3. creates the device file
4. initializes internal data structures
5. starts periodic monitoring

When the user loads the module using:

```
sudo insmod monitor.ko
```

the initialization logic runs automatically.

A crucial part of monitor.c is the ioctl handler. The ioctl handler receives requests from engine.c and interprets them. These requests can include:

- register a PID for monitoring
- unregister a PID
- possibly update or inspect limits

A conceptual glimpse is:

```
static long monitor_ioctl(struct file *file, unsigned int cmd, unsigned long arg)
{
    switch (cmd) {
        case MONITOR_IOCTL_REGISTER:
            ...
            break;
        case MONITOR_IOCTL_UNREGISTER:
```

```
        ...
        break;
    default:
        return -EINVAL;
  }
}
```

This switch-based handler makes it possible for user space to send structured control messages into the kernel module. The data itself is copied from user space using copy_from_user(), which is the safe Linux kernel mechanism for receiving user data.

Once a process is registered, the module periodically inspects it. For this, monitor.c uses a timer and/or workqueue mechanism. The timer periodically schedules monitoring logic so that memory checks happen repeatedly instead of only once.

The module then walks through the linked list of monitored processes and checks whether each process is still alive and how much resident memory it is using.

The central idea of the monitoring logic is:
- locate the task using its PID
- inspect memory usage
- compare with soft and hard limits
- log warning if soft limit crossed
- kill process if hard limit crossed
- remove stale entries if process no longer exists

The memory metric used in this project is RSS, or Resident Set Size. RSS reflects the amount of memory currently resident in physical RAM. It does not represent total virtual memory, but it is a useful measure of actual in-memory process footprint.

The enforcement logic can be explained in three stages.

First, if the process is not found anymore, it means the process has exited. In that case, the corresponding monitoring entry is stale and can be removed from the list.

Second, if the process is alive and its memory usage exceeds the soft limit, the module generates a warning. This warning is useful because it indicates that the process is consuming more memory than recommended, but it is not yet terminated.

Third, if the memory usage exceeds the hard limit, the module sends SIGKILL to the process. This ensures immediate termination.

Conceptually, the logic is:

```
if (rss > soft_limit && !soft_warned) {
    printk(KERN_INFO "soft limit exceeded...");
    soft_warned = 1;
}

if (rss > hard_limit) {
    send_sig(SIGKILL, task, 0);
    printk(KERN_INFO "hard limit exceeded, killing process...");
}
```

This is one of the most important parts of the entire project because it demonstrates kernel-space resource control. The runtime does not merely observe memory growth—it actively enforces policy.

The project output confirmed this behavior when the memory_hog workload allocated memory in 8 MB chunks until the process reached the configured hard limit. At that point, the kernel

module killed the process, and the runtime metadata reflected the termination as “killed, signal 9”.

This interaction between monitor.c and engine.c demonstrates user-kernel cooperation. The runtime decides what to launch and what limits to apply, while the kernel module applies continuous low-level monitoring and protection.

Another important aspect of monitor.c is cleanup. When the module is removed using rmmod, it must:
- stop its timer/workqueue
- destroy the device
- remove the class
- free internal list memory
- unregister device numbers

This is necessary because kernel modules operate in privileged space, and leftover resources can affect system stability.

The timer cleanup and synchronization are particularly important. During development, timer-related build compatibility issues had to be resolved by using the appropriate timer shutdown mechanism. This highlighted how kernel programming requires careful matching with the current kernel API.

In summary, monitor.c implements:
- character device creation
- ioctl request handling
- linked-list storage of monitored processes
- periodic memory inspection
- soft-limit warning generation
- hard-limit enforcement using SIGKILL

- device and memory cleanup on unload

Thus, monitor.c can be considered the enforcement engine of the project.

4.3 INTERACTION BETWEEN ENGINE.C AND MONITOR.C

The two files are deeply connected but operate at different privilege levels.

The interaction flow is as follows:

1. The supervisor in engine.c starts a container.
2. The runtime records the PID and requested memory limits.
3. The runtime opens /dev/container_monitor.
4. The runtime sends monitoring information through ioctl.
5. The kernel module receives the request and stores the process entry.
6. The kernel periodically checks the process.
7. If the process exceeds limits, the kernel logs a warning or kills it.
8. The supervisor later observes termination and updates metadata.

This cooperation is important because it reflects a real operating system architecture:
- user space manages execution policy and interaction
- kernel space manages protected enforcement and resource control

4.4 WHY THESE TWO FILES ARE THE CORE OF THE PROJECT

The entire project fundamentally depends on these two files.

Without engine.c:
- containers cannot be created
- workloads cannot run in isolated namespaces

- user commands cannot be processed
- logs cannot be captured
- metadata cannot be shown

Without monitor.c:
- memory usage cannot be enforced in kernel space
- no soft/hard limit behavior can be demonstrated
- the project would lack real resource protection

Therefore, engine.c and monitor.c together represent the two major dimensions of operating system behavior in this project:
- process and execution management
- memory monitoring and enforcement

This makes the code design both educational and practical, because it directly demonstrates how user-space and kernel-space components cooperate in a real operating system environment.

RESULT AND DISCUSSION

The results obtained from the implementation of the Jackfruit container runtime and memory monitoring system demonstrate the correct functioning of all major components, including container isolation, process management, logging, and kernel-level memory enforcement. Each task and workload was executed successfully, and the observed outputs validate the theoretical concepts of operating systems applied in this project.

The first stage of evaluation involved verifying the build process. The successful execution of the “make clean” and “make” commands resulted in the generation of all required binaries, including the runtime executable (engine), kernel module (monitor.ko), and workload programs (cpu_hog, io_pulse, and memory_hog). This confirmed that the development environment was correctly configured and that the codebase was compatible with the system’s kernel and compiler.

The next stage focused on root filesystem setup. The Alpine Linux root filesystem was successfully downloaded, extracted, and replicated into multiple directories representing different containers. The presence of workload binaries within these root filesystems confirmed that each container would have access only to its designated environment. This result validated the concept of filesystem isolation using chroot(), ensuring that processes inside containers operate independently from the host system.

The loading of the kernel module was then tested. The successful insertion of monitor.ko using the insmod command, along with the appearance of the module in lsmod output and the creation of the device file /dev/container_monitor, confirmed that the kernel module was properly registered. This result demonstrated correct kernel module initialization, device creation, and readiness for communication with user-space programs.

The supervisor runtime was then executed to manage containers. Upon running the supervisor command, the system initialized successfully and began listening for incoming commands.

This indicated that the runtime was capable of maintaining a persistent control process, which is essential for managing multiple containers. The correct functioning of the supervisor also confirmed that the control socket and metadata management structures were properly initialized.

The execution of the CPU-intensive workload (cpu_hog) provided insights into process scheduling and lifecycle management. The container executed the workload successfully, producing continuous output messages such as “alive” iterations. The process completed execution and exited normally with exit code 0. This result demonstrated that the runtime could correctly create containers, execute programs within isolated environments, and track their lifecycle. It also showed that CPU-bound processes are handled efficiently by the operating system scheduler.

The execution of the I/O-intensive workload (io_pulse) further validated the logging and inter-process communication mechanisms. The workload generated periodic output messages, which were successfully captured by the runtime and stored in log files. The logs command was able to retrieve and display this output correctly. This result confirmed that the pipe-based logging mechanism was functioning as expected, and that synchronization between producer and consumer components was correctly implemented.

The most critical result was observed during the execution of the memory-intensive workload (memory_hog). The workload incrementally allocated memory in fixed chunks, and the allocation messages were clearly visible in the logs. As the memory usage increased, the kernel module monitored the process in real time. When the memory usage exceeded the predefined hard limit, the process was terminated using a SIGKILL signal. This was reflected in the process status output, where the container was marked as “killed” with signal 9. This result confirmed that the kernel module was correctly tracking memory usage and enforcing limits, thereby demonstrating effective resource control at the kernel level.

In addition to workload execution, container lifecycle management was also tested. The start command successfully launched containers in the background, while the run command executed containers in the foreground and waited for their completion. The stop command was able to terminate running containers, and the ps command displayed accurate metadata for all containers, including their states, process IDs, and exit information. These results confirmed that the command-line interface and supervisor coordination were functioning correctly.

Another important observation was the behavior of the system under different workloads. CPU-bound processes continuously consumed processor time, while I/O-bound processes exhibited periodic execution patterns. The memory-bound process demonstrated controlled termination due to policy enforcement rather than natural completion. This highlights how the operating system manages different types of workloads and ensures fair resource allocation while maintaining system stability.

The logging mechanism also performed reliably across all scenarios. All outputs generated by container processes were successfully captured and stored, and no data loss or corruption was observed. This indicates that the bounded-buffer design and synchronization mechanisms were effective in handling concurrent logging operations.

Overall, the results obtained from this project confirm that the implemented system successfully achieves its objectives. The runtime is capable of creating and managing isolated containers, executing workloads, capturing logs, and enforcing memory constraints through kernel-level monitoring. The interaction between user space and kernel space is correctly established, demonstrating a clear understanding of operating system design principles.

The successful execution of all tasks, along with the correct behavior observed in different scenarios, validates the effectiveness of the implementation. The project not only meets the functional requirements but also provides a practical demonstration of key operating system concepts such as process isolation, scheduling, memory management, and system-level programming.

OUTPUTS

Screenshots_1_build_success.png

```
pratham-kumar@pratham-kumar-VMware-Virtual-Platform:~$ cd ~/OS-Jackfruit/boilerplate
pratham-kumar@pratham-kumar-VMware-Virtual-Platform:~/OS-Jackfruit/boilerplate$ make clean
if [ -d "/lib/modules/6.17.0-20-generic/build" ]; then make -C /lib/modules/6.17.0-20-generic/build M=/home/pratham-kumar/OS-Jackfruit/boilerplate clean; fi
make[1]: Entering directory '/usr/src/linux-headers-6.17.0-20-generic'
make[2]: Entering directory '/home/pratham-kumar/OS-Jackfruit/boilerplate'
  CLEAN   Module.symvers
make[2]: Leaving directory '/home/pratham-kumar/OS-Jackfruit/boilerplate'
make[1]: Leaving directory '/usr/src/linux-headers-6.17.0-20-generic'
rm -f engine memory_hog cpu_hog io_pulse *.o *.mod *.mod.c *.symvers *.order
rm -f *.log
rm -rf logs
rm -f /tmp/mini_runtime.sock
pratham-kumar@pratham-kumar-VMware-Virtual-Platform:~/OS-Jackfruit/boilerplate$ make
gcc -O2 -Wall -Wextra -o engine engine.c -lpthread
engine.c: In function ‘log_producer_thread’:
engine.c:519:9: warning: ‘__builtin_strncpy’ output may be truncated copying 31 bytes from a string of length 31 [-Wstringop-truncation]
  519 |         strncpy(item.container_id, id, sizeof(item.container_id) - 1);
      |         ^
engine.c: In function ‘start_container_locked.constprop’:
engine.c:714:5: warning: ‘__builtin_strncpy’ output may be truncated copying 31 bytes from a string of length 31 [-Wstringop-truncation]
  714 |     strncpy(rec->id, req->container_id, sizeof(rec->id) - 1);
      |     ^
engine.c:715:5: warning: ‘__builtin_strncpy’ output may be truncated copying 4095 bytes from a string of length 4095 [-Wstringop-truncation]
  715 |     strncpy(rec->rootfs, req->rootfs, sizeof(rec->rootfs) - 1);
      |     ^
engine.c:716:5: warning: ‘__builtin_strncpy’ output may be truncated copying 255 bytes from a string of length 255 [-Wstringop-truncation]
  716 |     strncpy(rec->command, req->command, sizeof(rec->command) - 1);
      |     ^
engine.c:725:5: warning: ‘__builtin_strncpy’ output may be truncated copying 31 bytes from a string of length 31 [-Wstringop-truncation]
  725 |     strncpy(cfg->id, rec->id, sizeof(cfg->id) - 1);
      |     ^
engine.c:726:5: warning: ‘__builtin_strncpy’ output may be truncated copying 4095 bytes from a string of length 4095 [-Wstringop-truncation]
  726 |     strncpy(cfg->rootfs, rec->rootfs, sizeof(cfg->rootfs) - 1);
      |     ^
engine.c:727:5: warning: ‘__builtin_strncpy’ output may be truncated copying 255 bytes from a string of length 255 [-Wstringop-truncation]
  727 |     strncpy(cfg->command, rec->command, sizeof(cfg->command) - 1);
      |     ^
engine.c:446:5: warning: ‘__builtin_strncpy’ output may be truncated copying 31 bytes from a string of length 31 [-Wstringop-truncation]
  446 |     strncpy(req.container_id, container_id, sizeof(req.container_id) - 1);
      |     ^
engine.c: In function ‘reap_children’:
engine.c:463:5: warning: ‘__builtin_strncpy’ output may be truncated copying 31 bytes from a string of length 31 [-Wstringop-truncation]
  463 |     strncpy(req.container_id, container_id, sizeof(req.container_id) - 1);
      |     ^
engine.c: In function ‘send_control_request’:
engine.c:1154:5: warning: ‘__builtin_strncpy’ output may be truncated copying 31 bytes from a string of length 31 [-Wstringop-truncation]
 1154 |     strncpy(req.container_id, container_id, sizeof(req.container_id) - 1);
      |     ^
gcc -O2 -Wall -static -o memory_hog memory_hog.c
gcc -O2 -Wall -static -o cpu_hog cpu_hog.c
gcc -O2 -Wall -static -o io_pulse io_pulse.c
make -C /lib/modules/6.17.0-20-generic/build M=/home/pratham-kumar/OS-Jackfruit/boilerplate modules
make[1]: Entering directory '/usr/src/linux-headers-6.17.0-20-generic'
make[2]: Entering directory '/home/pratham-kumar/OS-Jackfruit/boilerplate'
warning: the compiler differs from the one used to build the kernel
  The kernel was built by: x86_64-linux-gnu-gcc-13 (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0
  You are using:           gcc-13 (Ubuntu 13.3.0-6ubuntu2~24.04.1) 13.3.0
  CC [M]  monitor.o
  MODPOST Module.symvers
  CC [M]  monitor.mod.o
  CC [M]  .module-common.o
  LD [M]  monitor.ko
  BTF [M] monitor.ko
Skipping BTF generation for monitor.ko due to unavailability of vmlinux
make[2]: Leaving directory '/home/pratham-kumar/OS-Jackfruit/boilerplate'
```

Screenshots_2_rootfs_setup.png

```
pratham-kumar@pratham-kumar-VMware-Virtual-Platform:~/OS-Jackfruit$ wget https://dl-cdn.alpinelinux.org/alpine/v3.20/releases/x86_64/alpine-minirootfs
-3.20.3-x86_64.tar.gz
--2026-04-15 14:55:30--  https://dl-cdn.alpinelinux.org/alpine/v3.20/releases/x86_64/alpine-minirootfs-3.20.3-x86_64.tar.gz
Resolving dl-cdn.alpinelinux.org (dl-cdn.alpinelinux.org)... 151.101.194.132, 151.101.2.132, 151.101.66.132, ...
Connecting to dl-cdn.alpinelinux.org (dl-cdn.alpinelinux.org)|151.101.194.132|:443... connected.
HTTP request sent, awaiting response... 200 OK
Length: 3490290 (3.3M) [application/octet-stream]
Saving to: ‘alpine-minirootfs-3.20.3-x86_64.tar.gz’

alpine-minirootfs-3.20.3-x86_64.tar.g 100%[===================================================================>]   3.33M  16.8MB/s    in 0.2s

2026-04-15 14:55:30 (16.8 MB/s) - ‘alpine-minirootfs-3.20.3-x86_64.tar.gz’ saved [3490290/3490290]

pratham-kumar@pratham-kumar-VMware-Virtual-Platform:~/OS-Jackfruit$ mkdir rootfs-base
pratham-kumar@pratham-kumar-VMware-Virtual-Platform:~/OS-Jackfruit$ tar -xzf alpine-minirootfs-3.20.3-x86_64.tar.gz -C rootfs-base
pratham-kumar@pratham-kumar-VMware-Virtual-Platform:~/OS-Jackfruit$ ls rootfs-base
bin  dev  etc  home  lib  media  mnt  opt  proc  root  run  sbin  srv  sys  tmp  usr  var
pratham-kumar@pratham-kumar-VMware-Virtual-Platform:~/OS-Jackfruit$ cp -a rootfs-base rootfs-alpha
cp -a rootfs-base rootfs-beta
cp -a rootfs-base rootfs-gamma
pratham-kumar@pratham-kumar-VMware-Virtual-Platform:~/OS-Jackfruit$ cp -a rootfs-base rootfs-alpha
pratham-kumar@pratham-kumar-VMware-Virtual-Platform:~/OS-Jackfruit$ cp -a rootfs-base rootfs-beta
pratham-kumar@pratham-kumar-VMware-Virtual-Platform:~/OS-Jackfruit$ cp -a rootfs-base rootfs-gamma
pratham-kumar@pratham-kumar-VMware-Virtual-Platform:~/OS-Jackfruit$ cp boilerplate/cpu_hog rootfs-alpha/
pratham-kumar@pratham-kumar-VMware-Virtual-Platform:~/OS-Jackfruit$ cp boilerplate/io_pulse rootfs-beta/
pratham-kumar@pratham-kumar-VMware-Virtual-Platform:~/OS-Jackfruit$ cp boilerplate/memory_hog rootfs-gamma/
pratham-kumar@pratham-kumar-VMware-Virtual-Platform:~/OS-Jackfruit$ ls rootfs-base
bin  dev  etc  home  lib  media  mnt  opt  proc  root  run  sbin  srv  sys  tmp  usr  var
pratham-kumar@pratham-kumar-VMware-Virtual-Platform:~/OS-Jackfruit$ ls rootfs-alpha | grep cpu_hog
cpu_hog
pratham-kumar@pratham-kumar-VMware-Virtual-Platform:~/OS-Jackfruit$ ls rootfs-beta | grep io_pulse
io_pulse
pratham-kumar@pratham-kumar-VMware-Virtual-Platform:~/OS-Jackfruit$ ls rootfs-gamma | grep memory_hog
memory_hog
pratham-kumar@pratham-kumar-VMware-Virtual-Platform:~/OS-Jackfruit$
```

Screenshots_3_kernrl_module.png

```
pratham-kumar@pratham-kumar-VMware-Virtual-Platform:~/OS-Jackfruit$ cd boilerplate
pratham-kumar@pratham-kumar-VMware-Virtual-Platform:~/OS-Jackfruit/boilerplate$ sudo insmod monitor.ko
[sudo] password for pratham-kumar:
pratham-kumar@pratham-kumar-VMware-Virtual-Platform:~/OS-Jackfruit/boilerplate$ lsmod | grep monitor
monitor                12288  0
pratham-kumar@pratham-kumar-VMware-Virtual-Platform:~/OS-Jackfruit/boilerplate$ ls -l /dev/container_monitor
crw------- 1 root root 239, 0 Apr 15 15:00 /dev/container_monitor
pratham-kumar@pratham-kumar-VMware-Virtual-Platform:~/OS-Jackfruit/boilerplate$
```

Screenshots_4_supervisor_ready.png

```
pratham-kumar@pratham-kumar-VMware-Virtual-Platform:~/OS-Jackfruit/boilerplate$ lsmod | grep monitor
monitor                12288  0
pratham-kumar@pratham-kumar-VMware-Virtual-Platform:~/OS-Jackfruit/boilerplate$ ls -l /dev/container_monitor
crw------- 1 root root 239, 0 Apr 15 15:00 /dev/container_monitor
pratham-kumar@pratham-kumar-VMware-Virtual-Platform:~/OS-Jackfruit/boilerplate$ cd ~/OS-Jackfruit
pratham-kumar@pratham-kumar-VMware-Virtual-Platform:~/OS-Jackfruit$ sudo ./boilerplate/engine supervisor ./rootfs-base
Supervisor ready. base-rootfs=./rootfs-base control=/tmp/mini_runtime.sock
```

Screenshots_5_cpu_hog.png

```
pratham-kumar@pratham-kumar-VMware-Virtual-Platform:~/OS-Jackfruit$ cd ~/OS-Jackfruit
pratham-kumar@pratham-kumar-VMware-Virtual-Platform:~/OS-Jackfruit$ sudo ./boilerplate/engine start alpha ./rootfs-alpha /cpu_hog

container alpha started with pid 10966
pratham-kumar@pratham-kumar-VMware-Virtual-Platform:~/OS-Jackfruit$ sudo ./boilerplate/engine ps
ID      PID     STATE   STARTED SOFT_MIB        HARD_MIB        EXIT    ROOTFS
alpha   10966   exited  2026-04-15 15:06:38     40      64      code:0  ./rootfs-alpha
pratham-kumar@pratham-kumar-VMware-Virtual-Platform:~/OS-Jackfruit$ sudo ./boilerplate/engine logs alpha
cpu_hog alive elapsed=1 accumulator=17107564972555657481
cpu_hog alive elapsed=2 accumulator=14958271407607881607
cpu_hog alive elapsed=3 accumulator=11565067665932120422
cpu_hog alive elapsed=4 accumulator=11076698638966761843
cpu_hog alive elapsed=5 accumulator=11122054553243555228
cpu_hog alive elapsed=6 accumulator=16483789549138190986
cpu_hog alive elapsed=7 accumulator=7012601087838145749
cpu_hog alive elapsed=8 accumulator=6169772637089199617
cpu_hog alive elapsed=9 accumulator=1046958675129430401
cpu_hog alive elapsed=10 accumulator=5274159729215707858
cpu_hog done duration=10 accumulator=5274159729215707858
pratham-kumar@pratham-kumar-VMware-Virtual-Platform:~/OS-Jackfruit$
```

Screenshots_6_io_pulse.png

```
pratham-kumar@pratham-kumar-VMware-Virtual-Platform:~/OS-Jackfruit$ sudo ./boilerplate/engine start beta ./rootfs-beta /io_pulse
container beta started with pid 11194
pratham-kumar@pratham-kumar-VMware-Virtual-Platform:~/OS-Jackfruit$ sudo ./boilerplate/engine logs beta
io_pulse wrote iteration=1
io_pulse wrote iteration=2
io_pulse wrote iteration=3
io_pulse wrote iteration=4
io_pulse wrote iteration=5
io_pulse wrote iteration=6
io_pulse wrote iteration=7
io_pulse wrote iteration=8
io_pulse wrote iteration=9
io_pulse wrote iteration=10
io_pulse wrote iteration=11
io_pulse wrote iteration=12
io_pulse wrote iteration=13
io_pulse wrote iteration=14
io_pulse wrote iteration=15
io_pulse wrote iteration=16
io_pulse wrote iteration=17
io_pulse wrote iteration=18
io_pulse wrote iteration=19
io_pulse wrote iteration=20
pratham-kumar@pratham-kumar-VMware-Virtual-Platform:~/OS-Jackfruit$
```

Screenshots_7_memory_hog.png

```
io_pulse wrote iteration=20
pratham-kumar@pratham-kumar-VMware-Virtual-Platform:~/OS-Jackfruit$ sudo ./boilerplate/engine start gamma ./rootfs-gamma /memory_hog
container gamma started with pid 11326
pratham-kumar@pratham-kumar-VMware-Virtual-Platform:~/OS-Jackfruit$ sudo ./boilerplate/engine logs gamma
allocation=1 chunk=8MB total=8MB
allocation=2 chunk=8MB total=16MB
allocation=3 chunk=8MB total=24MB
allocation=4 chunk=8MB total=32MB
allocation=5 chunk=8MB total=40MB
allocation=6 chunk=8MB total=48MB
allocation=7 chunk=8MB total=56MB
allocation=8 chunk=8MB total=64MB
pratham-kumar@pratham-kumar-VMware-Virtual-Platform:~/OS-Jackfruit$ sudo ./boilerplate/engine ps
ID      PID     STATE   STARTED SOFT_MIB        HARD_MIB        EXIT    ROOTFS
gamma   11326   killed  2026-04-15 15:11:21     40      64      sig:9   ./rootfs-gamma
beta    11194   exited  2026-04-15 15:09:15     40      64      code:0  ./rootfs-beta
alpha   10966   exited  2026-04-15 15:06:38     40      64      code:0  ./rootfs-alpha
pratham-kumar@pratham-kumar-VMware-Virtual-Platform:~/OS-Jackfruit$
```

Screenshots_8_control_run.png

```
pratham-kumar@pratham-kumar-VMware-Virtual-Platform:~/OS-Jackfruit$ sudo ./boilerplate/engine ps
ID      PID     STATE   STARTED SOFT_MIB        HARD_MIB        EXIT    ROOTFS
gamma   11326   killed  2026-04-15 15:11:21     40      64      sig:9   ./rootfs-gamma
beta    11194   exited  2026-04-15 15:09:15     40      64      code:0  ./rootfs-beta
alpha   10966   exited  2026-04-15 15:06:38     40      64      code:0  ./rootfs-alpha
pratham-kumar@pratham-kumar-VMware-Virtual-Platform:~/OS-Jackfruit$ sudo ./boilerplate/engine start beta ./rootfs-beta /cpu_hog
container beta started with pid 11425
pratham-kumar@pratham-kumar-VMware-Virtual-Platform:~/OS-Jackfruit$ sudo ./boilerplate/engine stop beta
container beta already finished
pratham-kumar@pratham-kumar-VMware-Virtual-Platform:~/OS-Jackfruit$ sudo ./boilerplate/engine ps
ID      PID     STATE   STARTED SOFT_MIB        HARD_MIB        EXIT    ROOTFS
beta    11425   exited  2026-04-15 15:13:42     40      64      code:127        ./rootfs-beta
gamma   11326   killed  2026-04-15 15:11:21     40      64      sig:9   ./rootfs-gamma
beta    11194   exited  2026-04-15 15:09:15     40      64      code:0  ./rootfs-beta
alpha   10966   exited  2026-04-15 15:06:38     40      64      code:0  ./rootfs-alpha
pratham-kumar@pratham-kumar-VMware-Virtual-Platform:~/OS-Jackfruit$ sudo ./boilerplate/engine run delta ./rootfs-gamma /cpu_hog
container delta finished state=exited exit_code=127 exit_signal=0
pratham-kumar@pratham-kumar-VMware-Virtual-Platform:~/OS-Jackfruit$
```

GITHUB LINK (WITH STRUCTURE EXPLANATION)

The complete implementation of the project has been uploaded to the following GitHub repository:

GitHub Repository Link: https://github.com/PKumar041104/OS-Jackfruit

The repository has been carefully structured to ensure clarity, maintainability, and ease of evaluation. Only relevant source files and necessary documentation have been included, while unnecessary files such as compiled binaries, temporary files, and logs have been excluded using the .gitignore configuration.

The repository is organized into the following main components:

1. boilerplate/ directory

This directory contains all the core source code and build-related files required for the project.

• engine.c
This is the main user-space runtime file responsible for container creation, supervision, logging, and command handling.

• monitor.c
This file contains the Linux kernel module implementation for memory monitoring and enforcement.

• monitor_ioctl.h
This header file defines the ioctl interface used for communication between user space and kernel space.

• Makefile

The Makefile is used to compile both the user-space runtime and the kernel module. It ensures a structured and automated build process.

• cpu_hog.c, io_pulse.c, memory_hog.c
These files define the workload programs used to test CPU, I/O, and memory behavior respectively.

• environment-check.sh
This script is used to verify that the system environment is correctly configured for running the project.

The boilerplate directory represents the core implementation layer of the project.

2. screenshots/ directory

This directory contains all the execution screenshots required for submission. These screenshots serve as proof of successful implementation of each task.

The screenshots are named in a structured manner:

• Screenshots_1_build_success.png
• Screenshots_2_rootfs_setup.png
• Screenshots_3_kernel_module.png
• Screenshots_4_supervisor_ready.png
• Screenshots_5_cpu_hog.png
• Screenshots_6_io_pulse.png
• Screenshots_7_memory_hog.png
• Screenshots_8_control_run.png

This naming convention ensures a direct mapping between report figures and GitHub files, making it easy for evaluators to verify outputs.

3. docs/ directory

This directory is intended to store project documentation, including the final report. It ensures separation between source code and documentation files.

4. README.md

The README file provides a detailed explanation of the project, including setup instructions, execution steps, commands used, screenshots, and overall analysis. It acts as a guide for anyone who wants to understand or reproduce the project.

5. .gitignore

The .gitignore file ensures that unnecessary files such as compiled binaries (.o, .ko), temporary logs, and system-generated files are not included in the repository. This helps maintain a clean and professional project structure.

Overall, the repository structure reflects a modular and well-organized approach, separating code, outputs, and documentation. This makes the project easy to understand, reproduce, and evaluate. The structure also follows good software engineering practices by maintaining clarity, avoiding redundancy, and ensuring that only essential components are included.

REFERENCES / BIBLIOGRAPHY

1. Silberschatz, A., Galvin, P. B., and Gagne, G., Operating System Concepts, 10th Edition, Wiley.

2. Linux Manual Pages:
   - clone(2) — Linux process creation system call
   - chroot(2) — Change root directory
   - exec(3) — Execute program
   - waitpid(2) — Process termination handling

3. Linux Kernel Module Programming Guide
   Available at: https://tldp.org/LDP/lkmpg/2.6/html/

4. Alpine Linux Documentation
   Available at: https://alpinelinux.org/

5. GNU Make Documentation
   Available at: https://www.gnu.org/software/make/manual/

6. Official Linux Documentation
   Available at: https://www.kernel.org/doc/

7. Course Material:
   UE24CS242B - Operating Systems, PES University Lecture Notes

8. GitHub Repository (Project Reference):
   https://github.com/PKumar041104/OS-Jackfruit

APPENDIX - (Definitions, Acronyms, Abbreviations)

APPENDIX A: DEFINITIONS, ACRONYMS AND ABBREVIATIONS

Operating System (OS):
System software that manages hardware resources and provides services to applications.

Container:
A lightweight isolated execution environment that shares the host OS kernel.

Namespace:
A Linux feature that provides isolation of system resources such as process IDs, hostnames, and filesystems.

PID Namespace:
Provides isolation of process IDs so that processes inside a container have their own PID numbering.

UTS Namespace:
Allows containers to have independent hostnames and domain names.

Mount Namespace:
Provides isolation of filesystem mount points for containers.

chroot():
A system call that changes the root directory of a process, restricting its file access.

clone():
A system call used to create new processes with fine-grained control over resource sharing.

exec():

Replaces the current process image with a new program.

Process:

An instance of a program in execution.

Scheduler:

The OS component responsible for allocating CPU time to processes.

CPU-bound Process:

A process that primarily uses CPU resources.

I/O-bound Process:

A process that frequently performs input/output operations.

RSS (Resident Set Size):

The portion of a process's memory that is held in physical RAM.

Kernel Module:

A piece of code that can be dynamically loaded into the Linux kernel to extend its functionality.

SIGKILL:

A signal used to forcibly terminate a process.

IPC (Inter-Process Communication):

Mechanisms that allow processes to communicate and share data.

Pipe:

A unidirectional IPC mechanism used for communication between processes.

CLI (Command Line Interface):

A text-based interface used to interact with a system.

Makefile:

A file used to automate the build process of programs.

Virtual Machine (VM):

A software-based emulation of a physical computer system.

Root Filesystem (rootfs):

The top-level directory structure of a container or system.

Supervisor:

A long-running process that manages and monitors container execution.

Git:

A version control system used to track changes in source code.

GitHub:

A platform used for hosting and sharing code repositories.

THANK YOU